

and persists them in the GPU cache. RetrievalAttention can be adapted to utilize more complex static patterns (Li et al., 2024; Jiang et al., 2024), achieving the best trade-off between low inference cost and high accuracy. During the prefill phase, we physically separate the static tokens in the GPU memory with the remaining tokens, which are offloaded to the CPU memory indexed by the ANNS. To minimize data transfer over the slow PCIe interface, RetrievalAttention independently computes the attention results for the CPU and GPU components and then combines them, inspired by the FastAttention (Dao et al., 2022).

4 EVALUATION

In this section, we compare the performance of RetrievalAttention in long-context LLM inference against full attention and other state-of-the-art methods. Through experiments, we mainly explore the following questions: (1) **How does RetrievalAttention affect the model’s inference accuracy?** Specifically, we assess the generation accuracy of RetrievalAttention and other methods across various downstream tasks (§4.2) (2) **Can RetrievalAttention efficiently reduce the token generation latency of long-context LLM inference?** We compare the end-to-end decoding latency of RetrievalAttention with that of other baselines (§4.3).

4.1 EXPERIMENTAL SETUP

Testbed, Models, and Configurations. We conduct experiments on a server equipped with one NVIDIA RTX4090 GPU (24GB memory) and an Intel i9-10900X CPU with 10 physical cores (20 logical cores) and 128GB DRAM. The experiment results using NVIDIA A100 GPU can be found in §A.4. We implement RetrievalAttention on three state-of-the-art long-context LLMs, including Llama-3-8B-Instruct-262k (Gradient AI, 2024), Yi-6B-200K (01-ai, 2024a), and Yi-9B-200K (01-ai, 2024b). To show a practical speedup of RetrievalAttention and ensure the CPU memory consumption in long contexts does not exceed the DRAM capacity, we follow previous work (Tang et al., 2024) to run the benchmark in real-world single-batch scenarios.

Baselines. We compare RetrievalAttention with the following training-free baselines. (1) Full attention without KV cache as well as the version with KV cache using vLLM (Kwon et al., 2023). (2) StreamingLLM (Xiao et al., 2024b): it retains initial tokens along with fixed-length recent tokens in the GPU memory and discards remaining tokens. (3) SnapKV (Li et al., 2024): it only caches the critical tokens observed from the last window of the prompt. (4) InfLLM (Xiao et al., 2024a): it separates the KV cache of continuous token sequences into blocks and selects representative vectors for each block. In the decoding phase, the current query scans all representative vectors and retrieves top- k blocks with the highest similarity. (5) Quest (Tang et al., 2024): it keeps track of the minimal and maximal key values in KV cache pages and estimates the criticality of a page using the query vector. (6) InfiniGen (Lee et al., 2024): it prefetches only the essential KV cache entries by speculating important tokens required for subsequent attention layers.

To better assess the effectiveness of our method, we introduce two additional baselines using traditional vector search methods from Faiss (Douze et al., 2024). Specifically, Flat is an exact KNN method that performs a linear scan of all key-value vectors, whereas IVF indexes key vectors through clustering. By default, all indexing-based methods retrieve the top-100 nearest key vectors.

Benchmarks. We adopt three representative long-context benchmarks for evaluation.

- ∞ -Bench (Zhang et al., 2024b): this benchmark consists of 7 tasks, including three retrieval tasks (passKey retrieval, number retrieval, KV retrieval) and four realistic tasks (code debugging, math find, dialogue and multiple-choices questions). The average context length of ∞ -Bench is over 100K tokens.
- RULER (Hsieh et al., 2024): a comprehensive long-context benchmark consisting of 4 categories and 13 tasks, including retrieval, multi-hop tracing, aggregation, and QA tasks. The prompt length ranges from 4K to 128K, allowing us to determine the actual context window size of models.
- Needle-in-a-haystack (Greg Kamradt, 2023): it challenges the models to accurately retrieve information (the “needle”) hidden within a lengthy document (the “haystack”).

Table 2: Performance (%) of different methods and models on ∞ -Bench. The size of the static pattern is consistently 640 (128 initial tokens + 512 tokens in the local window). All indexing-based methods, including Flat, IVF, and RetrievalAttention retrieve top-100 key vectors by default. In the relatively complicated task KV Retrieval, we include the results of retrieving top-2000 key vectors.

	Methods	Act. Tokens	Retr.N	Retr.P	Retr.KV	Code.D	Math.F	En.QA	En.MC	Avg.
Llama-3-8B	FullAttention	128K	100.0	100.0	17.5	19.0	39.5	9.1	68.0	50.4
	StreamingLLM	2K	5.0	5.0	1.0	18.5	40.0	6.0	66.0	20.2 (-30.2)
	SnapKV	2K	100.0	100.0	0.5	18.0	40.0	11.8	67.0	48.2 (-2.2)
	InfLLM	640+2K	100.0	100.0	0.5	20.5	48.0	7.0	37.0	44.7 (-5.7)
	InfiniGen	2K	99.5	100.0	0.0	17.5	39.0	7.3	57.5	45.8 (-4.6)
	Quest	2K	100.0	100.0	0.0	18.0	40.0	8.2	67.0	47.6 (-2.8)
	Flat	640+100/2K	100.0	100.0	8.5/14.5	19.0	40.0	7.5	67.0	48.9 (-1.5) / 49.7 (-0.7)
	IVF	640+100/2K	94.0	100.0	9.5/14.0	19.0	40.0	7.8	67.0	48.2 (-2.2) / 48.8 (-1.6)
	RetrievalAttention	640+100/2K	100.0	100.0	9.0/14.0	19.0	40.0	7.5	67.0	48.9 (-1.5) / 49.6 (-0.8)
Yi-9B	FullAttention	128K	100.0	100.0	30.5	25.5	36.5	9.8	67.0	52.8
	StreamingLLM	2K	5.0	5.0	0.5	24.0	33.5	6.4	72.0	20.9 (-31.9)
	SnapKV	2K	63.0	100.0	0.5	23.0	33.0	10.3	68.5	42.6 (-10.2)
	InfLLM	640+2K	100.0	100.0	0.5	20.5	43.0	9.4	44.0	45.3 (-7.5)
	Quest	2K	99.0	100.0	0.0	22.5	34.5	10.4	68.5	47.8 (-5.0)
	Flat	640+100/2K	100.0	100.0	21.0/30.0	23.0	35.0	10.8	68.5	51.2 (-1.6) / 52.5 (-0.3)
	IVF	640+100/2K	99.0	100.0	19.5/29.5	23.0	35.0	10.7	69.0	50.9 (-1.9) / 52.3 (-0.5)
	RetrievalAttention	640+100/2K	99.5	100.0	20.0/30.0	23.0	35.0	9.5	68.5	50.8 (-2.0) / 52.2 (-0.6)
Yi-6B	FullAttention	128K	98.0	100.0	3.5	31.0	11.0	19.2	55.5	45.5
	StreamingLLM	2K	5.0	5.0	0.5	27.5	11.0	12.2	54.0	16.5 (-29.0)
	SnapKV	2K	39.0	100.0	0.0	30.5	8.5	17.1	55.0	35.7 (-9.8)
	InfLLM	640+2K	99.0	100.0	0.5	27.5	18.0	12.7	40.5	42.6 (-2.9)
	Quest	2K	98.5	100.0	0.0	30.5	8.5	17.3	54.5	44.2 (-1.3)
	Flat	640+100/2K	98.5	100.0	2.5/3.0	30.5	16.0	17.7	54.5	45.7 (+0.2) / 45.7 (+0.2)
	IVF	640+100/2K	98.0	100.0	2.5/3.5	29.5	16.0	17.5	54.5	45.4 (-0.1) / 45.6 (+0.1)
	RetrievalAttention	640+100/2K	95.0	99.0	3.0/3.0	30.0	16.0	17.6	54.5	45.0 (-0.5) / 45.0 (-0.5)

4.2 ACCURACY ON LONG CONTEXT TASKS

∞ -Bench. As shown in Table 2, RetrievalAttention achieves comparable accuracy to the full attention, benefiting from its efficient dynamic retrieval of important tokens. Static methods, such as StreamingLLM and SnapKV, lack this capability and, therefore, achieve sub-optimal accuracy. During token generation, the critical tokens change dynamically according to the current query, invalidating the previously captured static patterns. InfiniGen exhibits a noticeable drop in model accuracy compared to full attention due to inaccurate speculation of important tokens from previous layers. Although InfLLM and Quest supports dynamic retrieval of relevant blocks, it achieves nearly zero accuracy in complex tasks (i.e., KV retrieval) due to the low accuracy of representative vectors. Since RetrievalAttention can accurately identify the most relevant key vectors, it achieves the best accuracy in KV retrieval. Moreover, by retrieving more tokens (i.e., top-2000 shown in the column of Retr.KV) in KV retrieval, RetrievalAttention achieves nearly the same accuracy as full attention, which demonstrates the effectiveness of our method in complex and dynamic tasks.

It is worth noting that Flat and IVF need to scan 100% and 30% of the past key vectors to achieve the same task accuracy as RetrievalAttention. In contrast, RetrievalAttention only requires scan 1–3% vectors, resulting in much lower decoding latency.

RULER. Table 3 demonstrates that models utilizing RetrievalAttention achieve nearly the same task accuracy as full attention in different context lengths. In contrast, other training-free methods experience a noticeable reduction in accuracy, particularly for longer context sizes like 128K, as they fail to capture dynamically changed important tokens.

Needle-in-a-haystack. As shown in Figure 5, RetrievalAttention can effectively focus on information at various positions across different context windows, ranging from 4K to 128K. In contrast, other methods like StreamingLLM encounter difficulties when critical information lies beyond the range of the static patterns, whose results are shown in §A.2.

Table 3: Performance (%) of different methods and models on RULER.

	Methods	Act. Tokens	Claimed	Effective	4K	8K	16K	32K	64K	128K	Avg.
Llama-3-8B	FullAttention	128K	262K	32K	93.13	90.49	89.27	85.11	82.51	78.74	86.54
	StreamingLLM	2K	-	<4K	60.10	28.77	20.99	16.36	12.52	11.34	25.01 (-61.53)
	SnapKV	2K	-	4K	91.51	80.70	75.53	70.84	65.44	58.68	73.78 (-12.76)
	InfLLM	640+2K	-	4K	85.20	52.86	38.29	32.44	27.94	25.71	43.74 (-42.81)
	Flat	640+100	-	16K	92.71	87.93	87.01	84.97	80.99	74.34	84.66 (-1.89)
	IVF	640+100	-	16K	92.73	87.86	87.22	84.74	78.46	68.21	83.20 (-3.34)
	RetrievalAttention	640+100	-	16K	92.64	88.46	86.80	84.78	80.50	74.70	84.70(-1.85)
Yi-9B	FullAttention	128K	200K	8K	91.02	86.62	82.85	73.17	67.08	60.51	76.87
	StreamingLLM	2K	-	<4K	57.53	28.30	19.08	13.48	12.53	12.81	23.95 (-52.92)
	SnapKV	2K	-	4K	90.39	75.59	64.48	48.70	39.28	32.97	58.57 (-18.30)
	InfLLM	640+2K	-	<4K	82.66	50.36	36.17	28.20	22.65	20.94	40.16 (-36.71)
	Flat	640+100	-	8K	91.09	87.71	84.42	74.58	66.16	59.50	77.24 (+0.37)
	IVF	640+100	-	8K	91.03	91.04	83.85	72.19	65.13	58.04	76.29 (-0.58)
	RetrievalAttention	640+100	-	8K	90.78	86.32	82.95	73.73	65.67	59.15	76.43(-0.44)
Yi-6B	FullAttention	128K	200K	<4K	84.52	77.77	69.12	61.64	58.36	55.77	67.86
	StreamingLLM	2K	-	<4K	51.66	24.57	15.82	9.70	9.77	11.54	20.51 (-47.35)
	SnapKV	2K	-	<4K	80.94	59.55	45.36	36.11	33.43	29.53	47.49 (-20.37)
	InfLLM	640+2K	-	<4K	76.42	44.38	34.11	27.11	25.28	25.33	38.77 (-29.09)
	Flat	640+100	-	<4K	83.69	77.26	67.28	60.58	57.27	50.63	66.12 (-1.75)
	IVF	640+100	-	<4K	83.25	76.90	67.00	58.94	55.99	50.31	63.33 (-2.53)
	RetrievalAttention	640+100	-	<4K	83.01	76.56	67.49	59.46	57.20	51.44	65.86(-2.00)

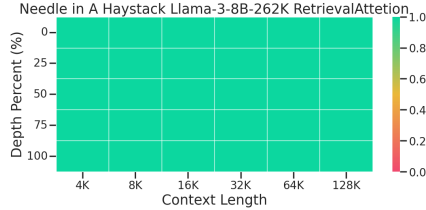


Figure 5: Performance of RetrievalAttention in Needle-in-a-haystack.

Methods	4K	8K	16K	32K	64K	128K
Full (without cache)	0.527	1.167	2.672	6.214	15.263	43.927
vLLM	OOM	OOM	OOM	OOM	OOM	OOM
StreamingLLM	0.029	0.030	0.029	0.030	0.030	0.029
SnapKV	0.029	0.028	0.028	0.029	0.029	0.028
InfLLM	0.058	0.063	0.063	0.065	0.067	0.069
Flat	0.140	0.178	0.226	0.328	0.522	0.922
IVF	0.128	0.140	0.162	0.201	0.253	0.373
RetrievalAttention	0.137	0.144	0.156	0.162	0.169	0.188

Table 4: Per-token generation latency (s) as context length varies from 4K to 128K on Llama-3-8B.

4.3 DECODING LATENCY

As the context length increases, the decoding latency of full attention significantly increases due to its quadratic time complexity. Enabling the KV cache (vLLM) incurs out-of-memory (OOM) issues due to limited GPU memory. The latency of StreamingLLM, SnapKV, and InfLLM remains relatively stable because of constant tokens involved in the attention computation, but they suffer significant model accuracy degradation. Due to efficient attention-aware vector search, RetrievalAttention achieves $4.9\times$ and $1.98\times$ latency reduction compared to Flat and IVF for the 128K context.

Table 5 presents the breakdown of end-to-end latency for different retrieval attention-based algorithms under the 128K context length. RetrievalAttention only requires 34.0% of the time for vector search, while Flat and IVF spend 86.6% and 67.0% of time, respectively. This is because RetrievalAttention scans less data for a high recall, avoiding memory bandwidth contention when multiple heads are performing parallel retrieval on the CPU side. Overall, compared with Flat and IVF, RetrievalAttention effectively reduces the index search latency by 91% and 74%, respectively. This advantage becomes more pronounced with longer context lengths.

4.4 INDEX RECALL VS. SCANNING VECTORS

Now, we conduct a micro-analysis of the efficiency of attention-aware vector search by examining the relationship between recall and the number of scanned key vectors. The number of key vectors scanned to achieve a target recall serves as an indicator of search efficiency. Figure 6 demonstrates that for the Q to K search, RetrievalAttention requires scanning only a very limited number of key vectors (1–3%) to reach a recall rate higher than 0.95, whereas traditional indexes necessitate retrieving a significantly higher number of keys. We also included a well-known OOD-optimized solution RobustVamana (Jaiswal et al., 2022) for comparison. However, it performs poorly on